Introduction to SSH

SSH 入门

Start Exploring→

Let’s Assume? 假设一下?

Suppose you need to run code on a **more powerful professional workstation** in the lab

假设你需要在实验室里一台**性能更强的专业工作站**上跑代码

OR

Need to debug your coursework on the school’s **Linux server**

需要在学校的**Linux服务器**上调试写好的作业

(CS major assignments like PGA, OSC, CPP usually need to run on the school's Linux server)
(CS专业课如PGA, OSC, CPP等作业均需运行在学校的Linux服务器上)

But... 但是...

- You can’t run to the lab for every single issue
  你不能什么问题都跑到实验室
- Compilation results differ between Windows and Linux
  Windows 和 Linux 编译结果不一致

Remote Desktop? 远程桌面?

You might have used:

- **ToDesk**
- **Sunlogin (向日葵)**
- **UU Remote (UU远程)**
- **RustDesk**
- **X2Go**

More professional ones:

- **Windows Remote Desktop (RDP)**
  Windows 远程桌面 (RDP)

Features:

- Directly view B’s **screen** on computer A
  在 A 电脑上直接看到 B 电脑的**屏幕画面**
- Click icons on computer B with your mouse
  用鼠标点击 B 电脑的图标
- Just like sitting in front of computer B
  就像坐在 B 电脑面前一样

What if... 如果...

IT: Hehe 😁 IT: 嘿嘿 😁

For security reasons, school IT disables these remote desktop tools on the university network

出于安全考虑，学校IT在学校网络下禁用这些远程桌面工具

But! You still need to operate the server remotely. **What to do?**

但是！你仍然需要远程操作服务器怎么办?

SSH: A basic but secure remote tool!

SSH: 一个基础但安全的远程工具!

So, what is SSH? 那什么是 SSH?

- **SSH (Secure Shell)** is also a type of remote operation

  SSH (Secure Shell) 也是一种远程操作

- But it **does not transmit the screen** (usually)

  但是它**不传输画面**（通常情况下）

- It transmits **text commands**

  它传输的是**文字指令**

- You control computer B’s **command line (Terminal)**

  你控制的是 B 电脑的**命令行 (Terminal)**

Core Functions of SSH SSH 的核心功能

- **Remote Login** (Access another computer’s system)
  远程登录 (进入另一台电脑的系统)
- **Remote Command Execution** (Make another computer work)
  远程执行命令 (让另一台电脑干活)
- **Secure File Transfer** (Upload code, download logs)
  安全传输文件 (把代码传上去，把日志拉下来)

Why "Secure Shell"? 为什么叫 "Secure Shell"?

- **Shell** = Command Line 命令行
- **Secure** = Encrypted Transmission 加密传输

All SSH communications are encrypted and cannot be easily intercepted.

SSH 的所有通信都是加密的，不会被轻易截获。

SSH: The Perfect Solution SSH：完美的解决方案

You need a way to:

你需要一种方式：

1. **Operate the lab server from your dorm computer**

   可以在宿舍电脑操作机房的服务器

2. **Be secure and stable**

   安全且稳定

3. **Open files on the remote server with VS Code**

   能用 VS Code 打开远程服务器上的文件

SSH = Control another computer remotely from your local machine

SSH = 让你在本地电脑上远程控制另一台电脑

III. How to use SSH? 如何使用 SSH?

1. Traditional Command Line SSH

1. 传统命令行 SSH

SSH is a traditional command line tool usable in the system terminal

SSH 是传统的命令行工具，可以在 系统终端 里使用

```
ssh user@remote-server-ip
```

2. Graphical SSH — VS Code Remote SSH

2. 图形化 SSH —— VS Code Remote SSH

Tools Preparation

工具准备

- **VS Code** (Visual Studio Code)
- Extension: **Remote - SSH** 插件
- A remote Linux server 一台远程 Linux 服务器
  - IP Address IP 地址
  - Username 用户名
  - Password (or SSH Private Key) 密码（或 SSH 私钥）

IV. Start Connecting 开始连接 —— VS Code Remote SSH

1. Install Extension

1. 安装插件

- Open VS Code Extensions
  打开 VS Code Extensions
- Search for **"Remote - SSH"**
  搜索 **"Remote - SSH"**
- Click Install
  点击 Install

2. Open Connection Entry

2. 打开连接入口

- Click the green icon at the bottom left (><)
  点击左下角绿色图标 (><)
- Select **"Connect to Host..."**
  选择 **"Connect to Host..."**

3. Configure Host

3. 配置 Host

Enter server address:

输入服务器地址:

```
user@ServerIP
# Example: student@192.168.10.35
```

4. Enter Password

4. 输入密码

- After successful login, bottom left shows "SSH: hostname"
  登录成功后，左下角显示 "SSH: hostname"

V. Using VS Code on Remote Server 在远程服务器上使用 VS Code

Once connected, it’s just like being local!

连接成功后，就像在本地一样！

1. Open Remote Folder

1. 打开远程文件夹

- File Explorer -> Open Folder -> `/home/student/project/`

2. Remote Editing

2. 远程编辑

- Edited code saves automatically to the server

编辑代码自动保存到服务器

3. Built-in Terminal

3. 内置终端

- VS Code Terminal is the **Remote Server's Shell**

VS Code Terminal 是**远程服务器的 Shell**

```
python3 app.py
gcc main.c -o main
ls
```

VI. Make SSH More Secure — SSH Key 让 SSH 更安全 —— SSH Key

Public Key and Private Key

公钥与私钥

SSH Key is a pair of keys:

SSH Key 是一对钥匙：

Private Key (私钥)

- Saved on your computer
  保存在你的电脑
- **Never leak this**
  绝对不能泄露

Public Key (公钥)

- Placed on server/GitHub
  放在服务器/GitHub
- Used for identity verification
  用于验证身份

Benefits: Password-less login, more secure, faster

好处：免密码登录，更安全、快捷

VII. GitHub Configure SSH Key GitHub 配置 SSH Key

1. Generate SSH Key

1. 生成 SSH Key

Run in Terminal (Terminal / Git Bash):

在终端 (Terminal / Git Bash) 运行：

```
ssh-keygen # Quickly generate an SSH key pair / 快速生成一个ssh密钥对
```

2. Find Key

2. 找到 Key

Open `~/.ssh/` directory:

打开 `~/.ssh/` 目录：

- `id_ed25519` (Private Key / 私钥)
- `id_ed25519.pub` (Public Key / 公钥) -> **Copy the content of this file / 复制这个文件的内容**

VII. GitHub Configure SSH Key GitHub 配置 SSH Key

3. Upload to GitHub

3. 上传到 GitHub

1. GitHub -> **Settings**
2. **SSH and GPG keys** -> **New SSH key**
3. Paste public key content
   粘贴公钥内容

4. Test Connection

4. 测试连接

```
ssh -T git@github.com
```

> "Hi username! You’ve successfully authenticated..."

Reference: Connecting to GitHub with SSH

VIII. Summary 八、总结

- **What is SSH**

  SSH 是什么

  - Secure remote operation protocol 安全的远程操作协议

- **Why needed**

  为什么需要

  - Remote development, server management 远程开发、管理服务器

- **VS Code Remote SSH**

  VS Code Remote SSH

  - Graphical, one-click remote development 图形化、一键远程开发

- **SSH Key**

  SSH Key

  - Password-less login, GitHub authentication 免密登录、GitHub 身份验证

Thank you everyone!

谢谢大家！